

Streaming Self-Improvement of Language Models via Online Self-Distillation from Verifiable Feedback

Anonymous authors
Paper under double-blind review

Abstract

Deployed language-model agents face a stream of tasks and receive only sparse, verifiable feedback—a unit test passes, an answer matches, a query executes. We study how a model can permanently improve from this signal alone, online, without ground-truth labels or a stronger teacher. We propose an online self-distillation method: each incoming problem is answered and scored before any update; verified successes enter a memory that serves both retrieval-based in-context learning and, crucially, self-distillation—the model conditioned on its own verified answer acts as teacher for the same model without it, distilling the answer’s information into the weights. A forward-looking re-evaluation pass lets the improving model recover previously failed problems, converting weight gains back into training data. On seven diverse StreamBench tasks (code, SQL, diagnosis, QA, tool use), our method beats zero-shot, streaming-ICL, and REINFORCE++ baselines on every task. Under distribution shift—three domains interleaved into one stream—a single online-distilled model matches or beats three per-domain specialists, with positive cross-domain transfer to code (+3.4 points), while retrieval self-segregates (99.9% same-domain), showing the transfer lives in the weights; the RL baseline collapses on the same stream. Finally, comparing online training to multi-epoch batch self-distillation on identical data, the online-trained model masters the source data better (+5.5 points) [TODO: and transfers better to an unseen domain (results pending)]. Together these results position online self-distillation as a simple, stable recipe for continual self-improvement from verifiable feedback.

1 Introduction

Language models deployed as agents encounter their tasks as a stream: one problem at a time, in an order they do not control, with feedback that is sparse but verifiable—a unit test passes, a SQL query returns the right rows, a diagnosis matches the record. Two families of methods exploit this signal today. Memory-based approaches such as streaming in-context learning retrieve past verified successes as few-shot demonstrations; they adapt instantly but the model itself never improves, and gains vanish when memory is unavailable. Reinforcement learning from verifiable rewards updates the weights, but a scalar reward per attempt is a thin learning signal, and—as we show—policy optimization can collapse catastrophically on long heterogeneous streams.

We study a third route: online self-distillation. When the model solves a streamed problem and the oracle verifies it, the model conditioned on its own verified answer becomes a teacher for the same model without that answer. Matching the teacher’s hint-informed next-token distribution moves the answer’s information into the weights—a dense, token-level signal derived entirely from the model’s own verified output. No gold labels, no external teacher, no preference pairs.

Our method (§2) couples this distillation step with a reward-filtered memory used for k NN in-context retrieval, and a forward-looking re-evaluation pass: problems the model failed on arrival are periodically retried with the current, improved model; late successes enter memory and become training signal. This closes a flywheel—weight gains produce new

verified data, which produces further weight gains—that pure memory methods and pure RL both lack. Scoring always precedes any use of a problem for training, so the streaming metric is strictly predict-then-train.

Contributions.

- A simple online self-distillation recipe for streaming self-improvement from verifiable 0/1 feedback only (§2).
- State-of-the-art streaming accuracy on all seven StreamBench tasks spanning code, SQL, medical diagnosis, multi-hop QA, and tool use, against zero-shot, streaming-ICL, and REINFORCE++ baselines (§3.1).
- Robustness to distribution shift: on a stream interleaving three domains, one shared model matches or beats three per-domain specialists, with positive cross-domain transfer to code; retrieval self-segregates by domain (99.9%), so the transfer is carried by the weights. The RL baseline collapses on the same stream (§3.2).
- Online beats batch: trained on identical problems with the same loss and model, the online-trained model masters the source data better than multi-epoch batch self-distillation (+5.5 points) [TODO: and transfers better to an unseen target stream] (§3.3).

2 Method: Online Self-Distillation from Verifiable Feedback

Setting. Problems $x_1, x_2, \dots$ arrive in a fixed stream in batches of $B=10$. For each problem the environment provides a binary oracle $r(x, y) \in \{0, 1\}$ (test execution, exact match, label equality). Gold answers exist only inside the oracle; only the bit leaves it. The streaming metric is prequential: every problem is answered and scored before any update that involves it. We maintain a policy π_θ (base model + LoRA), a frozen copy of the base model π_{ref} , and a memory $M : x \mapsto (y^*, e)$ storing each problem’s latest verified answer and an embedding of its question.

Per-batch loop. Each stream batch triggers three phases (Algorithm 1):

- (1) Score. For each new x : retrieve the $k=3$ nearest past problems from M (embedding k NN, excluding x) as in-context demonstrations, generate greedily, and query the oracle. The result is recorded—permanently—and, if correct, the answer is stored in M (reward filter).
- (2) Forward re-evaluation. Problems from the last $W=9$ batches are re-answered with the current model and re-scored. Successes update M : failures recovered by the improved model enter memory late, and existing entries are refreshed into the current model’s phrasing (keeping distillation targets near on-policy). Re-evaluation never touches recorded scores.
- (3) Self-distillation. For each window problem x with $M(x) = y^*$: the student sees prompt $P(x) = \text{system} + \text{demos} + x$; the teacher sees $P(x)$ followed by y^* as an assistant turn and x repeated. A completion $\hat{y}$ is generated from the teacher prefix, and we minimize the token-level forward KL

$$\mathcal{L}(\theta) = \sum_t \text{KL}(\pi_{\text{ref}}(\cdot \mid P^+(x), \hat{y}_{<t}) \parallel \pi_\theta(\cdot \mid P(x), \hat{y}_{<t})), \quad (1)$$

where P^+ denotes the hint-augmented prompt. The teacher is the frozen base model—the only asymmetry is the hint. The student learns to produce the teacher’s hint-informed distribution without the hint, compressing the verified answer into the weights. We use 2 epochs per window pool, LoRA ($r=16$, $\alpha=32$, all linear layers), learning rate 5×10^{-5} , and skip the first 3 completion tokens.

Why no leakage. The hint is always the model’s own earlier oracle-passing output, never a dataset field; demos always come from strictly earlier batches; and a problem’s score is final before it can enter M . We audit these causal-ordering invariants programmatically over the actual run logs (Appendix A).

Algorithm 1 Online SDFT (one stream batch)

```

1: for each new problem  $x$  in batch  $B_t$  do
2:    $y \leftarrow \pi_\theta(\text{system} + \text{kNN}_3(M, x) + x)$  (greedy)
3:   record  $r(x, y)$  (prequential metric; final)
4:   if  $r(x, y) = 1$  then
5:      $M[x] \leftarrow y$ 
6:   end if
7: end for
8: for each  $x$  in last  $W$  batches do
9:    $y' \leftarrow \pi_\theta(\text{system} + \text{kNN}_3(M, x) + x)$ 
10:  if  $r(x, y') = 1$  then
11:     $M[x] \leftarrow y'$  (recover / refresh)
12:  end if
13: end for
14: for each  $x$  in window pool with  $M[x] = y^*$  do
15:   $\hat{y} \sim \pi_\theta(P^+(x))$ ; update  $\theta$  by Eq. 1
16: end for

```

Generation source. Conditioning the generated completion $\hat{y}$ on the hint (vs. sampling it from the bare student prompt, as in offline self-distillation) turns out to be a minor, bootstrap-phase choice: the two settings differ by ~ 1 point and only early in the stream (§3.4), so our results do not hinge on it.

3 Experiments

Common setup. All experiments use Qwen2.5-Coder-7B-Instruct with LoRA ($r=16$), greedy decoding, batch size 10, window $W=9$, $k\text{NN}$ $k=3$ (Qwen3-Embedding-0.6B), seed-42 stream order identical across methods. Baselines: Base (zero-shot, no adaptation), ICL (Self-StreamICL: reward-filtered memory + $k\text{NN}$ demos, frozen weights), and REINFORCE++ (policy gradient with global-whitened advantages, PPO clipping, KL to the frozen base, replaying stored completions over the same rolling window). All methods see identical problems in identical order; base, ICL, and REINFORCE++ use exactly one oracle call per problem (cost accounting for SDFT in §3.4).

3.1 Seven streaming benchmarks

Table 1 reports each benchmark’s native accuracy over its full stream. Our method attains the best accuracy on all seven tasks, spanning competitive code generation, data-science code, two text-to-SQL tasks, medical diagnosis, multi-hop QA, and function calling. Two patterns recur. First, ICL is a strong baseline where retrieval finds near-duplicates (DDX-Plus) but can hurt on tasks where retrieved demonstrations mislead (HotpotQA: 0.509 vs base 0.521); self-distillation recovers and surpasses base in exactly those cases (0.562), because the signal is consolidated in weights rather than recalled at inference. Second, REINFORCE++ is inconsistent—above ICL on some tasks, far below base on others (Spider 0.587 vs 0.747)—previewing the stability failure of §3.2.

3.2 Distribution shift: one model, three domains

We interleave DS-1000 (code, 955), DDXPlus (diagnosis, 1,764), and HotpotQA (QA, 1,500) into a single 4,219-problem stream by a random merge that preserves each domain’s internal order, making per-domain slices exactly comparable to the standalone runs. Table 2 shows the outcome.

Three findings. (i) One model $\geq$ three specialists: the fused model beats the aggregated standalone runs overall (.585 vs .573) and is no worse on any domain, with positive cross-domain transfer to code (+3.4 points on DS-1000). (ii) Transfer lives in the weights: retrieval self-segregates—99.9% of retrieved demonstrations are same-domain (2 of 2,059 queries pull

Table 1: Streaming accuracy on seven StreamBench tasks (native metric per task; identical stream order and model for all methods). Our method is best on all seven.

Benchmark	Domain	Size	Base	ICL	R++	Ours
LiveCodeBench	code	1055	0.338	0.424	0.386	0.448
DS-1000	Python	955	0.325	0.367	0.369	0.423
Spider	text-to-SQL	2147	0.747	0.778	0.587	0.808
DDXPlus	medical dx	1764	0.436	0.647	0.404	0.664
BIRD	hard SQL	1534	0.330	0.346	0.271	0.354
HotpotQA	multi-hop QA	1500	0.521	0.509	0.537	0.562
ToolBench	func-calling	750	0.549	0.593	0.607	0.615

Table 2: Fused-stream results (4,219 problems). “SDFT standalone” aggregates the three per-domain runs at matched prefixes (regret summed, accuracy weighted). One online-distilled model matches or beats three specialists.

Method	Acc	Cum. regret	DS-1000	DDXPlus	HotpotQA
Base	0.439	2365	0.318	0.436	0.521
ICL $k=3$	0.533	1971	0.358	0.647	0.509
REINFORCE++ (fused)	0.253	3153	0.213	0.262	0.267
SDFT standalone ($3\times$)	0.573	1800	0.423	0.664	0.562
SDFT fused (ours)	0.585	1750	0.457	0.676	0.560

any cross-domain demo)—so prompts are effectively domain-pure and cannot explain the gains. (iii) RL collapses: REINFORCE++ tracks ~ 5 points below SDFT for 2,000 problems, then diverges catastrophically (approximate KL turns negative, clip fraction spikes $0.08 \rightarrow 0.68$, advantage std triples); accuracy falls to zero for the remainder of the stream (final 0.253). The same hyperparameters are stable on every shorter standalone stream, indicating a long-horizon instability that online self-distillation simply does not exhibit: its per-window KL objective is anchored to a frozen teacher and its rolling accuracy climbs monotonically ($.44 \rightarrow .68$ peak) over the same stream (Figure 1).

3.3 Online vs. batch training

Does the online schedule itself matter, beyond the loss? We compare our online run on DS-1000 against batch self-distillation: the identical trainer, loss, model, and oracle, but run as classic offline epochs (4 passes over all 955 problems; per pass: generate, oracle-filter, distill on verified answers; no memory, no stream). We then evaluate both resulting adapters with a single bare-prompt greedy pass over the full problem set (no demonstrations, no memory).

Table 3: Source-data mastery after training on DS-1000 (bare-prompt greedy pass, all 955 problems).

Model	Acc	Solved / 955
Base	0.313	313
Batch SDFT (4 epochs)	0.425	406
Online SDFT (ours)	0.480	458

The online-trained model solves 458/955 (.480) versus 406 (.425) for batch—a +5.5-point gap on the very data batch trained on for four epochs—and its advantage comes from breadth: 123 problems only online solves vs 71 only batch solves, with equal (small) forgetting relative to base. We attribute the gap to the stream’s implicit curriculum: each problem is revisited across a rolling window while the model evolves, and forward re-evaluation keeps converting newly in-reach problems into training signal, whereas single-sample batch generation can only ever train on what the current model already solves. [TODO: Batch with $n=5$ sam-

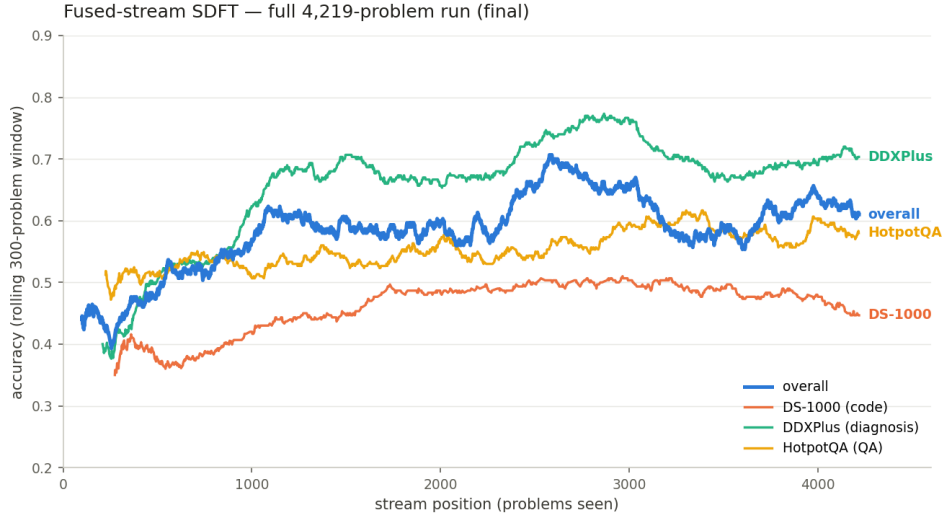


Figure 1: Rolling accuracy (300-problem window) of online SDFT over the fused three-domain stream, overall and per domain. All domains improve simultaneously; no interference from distribution shift.

pled attempts per problem (running) tests whether exploration closes this gap.] [TODO: Transfer: both adapters warm-start online SDFT on the unseen HotpotQA stream; compare accuracy/regret vs from-scratch (.562/657). Runs pending.]

3.4 Ablations and cost

Generation source. Sampling the distillation completion from the bare student prompt (offline-SDFT default) vs the hint-conditioned prefix (ours) changes DS-1000 by +1.3 points in favor of the former, with the difference confined to the early stream; on the fused stream the two are even after a small early deficit. The choice is a bootstrap-phase detail, not a driver. Epochs. 4 inner epochs beat 2 on DS-1000 (+21 cumulative-regret at matched pressure); learning-rate increases do not substitute for epochs. Multi-sample distillation. Distilling from $N=10$ sampled candidates (filtered or not) matches single-sample distillation—sample count is not the bottleneck. Oracle budget. Our method issues ~ 9.8 oracle calls per problem (1 scored on arrival + ~ 8.8 memory-only re-evaluations); baselines use 1. The extra calls only gate memory; they never alter recorded scores. Giving ICL the same budget would not help—a frozen model re-answering with the same distribution recovers almost nothing—and REINFORCE++ at 1 call/problem already shows that training-with-feedback alone does not close the gap.

4 Analysis: Why Online Self-Distillation Works

The flywheel. The method’s components are individually simple but mutually reinforcing: weight updates improve the model; forward re-evaluation lets the improved model recover problems it failed on arrival (84 of 1,463 memory entries in the fused run entered late this way); recovered problems become demonstrations and distillation targets; and fresher targets keep the KL objective near on-policy. A frozen-weights method (ICL) receives none of this—re-answering with an unchanged model recovers almost nothing—and pure RL receives the reward but not the dense token-level signal.

Distillation as supervision, memory as scaffolding. Wherever ICL hurts (HotpotQA, misleading neighbors), self-distillation still helps: the hint channel injects only the model’s own verified answer for the same problem, a signal that cannot mislead. Conversely, wherever

ICL is strong (DDXPlus), consolidation into weights preserves the gains while freeing the context window.

Stability. The distillation loss (Eq. 1) is anchored to a frozen reference teacher and its targets are always oracle-verified text. There is no advantage estimate, no clipped ratio, no moving critic—the failure surfaces that destroyed REINFORCE++ on the long fused stream simply do not exist. Across $\sim 12,000$ streamed problems in this paper, online SDFT never regressed catastrophically.

Predict-then-train integrity. Because every reported number is produced by a model that had never seen that problem—scored before storage, demonstrations strictly from the past, hints strictly self-generated—the streaming metric doubles as a continual-generalization measure. We release a causal-ordering audit that verifies these invariants over the raw run logs (Appendix A).

5 Related Work

Streaming adaptation and memory. StreamBench (Wu et al., 2024) formalizes online LLM improvement under prequential evaluation and shows Self-StreamICL—reward-filtered memory retrieved as demonstrations—is a strong baseline; our ICL baseline is exactly this method. Memory-augmented agents more broadly (e.g., Madaan et al., 2022) adapt behavior without weight updates and thus remain bounded by the frozen model’s ability.

Self-distillation for LLMs. Distilling a model conditioned on privileged information into itself without that information underlies self-distillation fine-tuning (Yang et al., 2024) and generalized knowledge distillation (Agarwal et al., 2024); SDPO (Hübotter et al., 2026) brings feedback-conditioned self-teachers into policy optimization. These methods are offline or rollout-batch; our contribution is the online instantiation—verified-answer hints harvested from a stream, coupled with memory and forward re-evaluation—and the finding that the online schedule itself improves both mastery and **[TODO: transfer]**.

RL from verifiable rewards. GRPO/RLOO/REINFORCE++ (Shao et al., 2024; Ahmadian et al., 2024; Hu, 2025) optimize scalar verifiable rewards and power modern reasoning models, but operate on thin per-attempt signal; our experiments show a faithful REINFORCE++ implementation is unstable on long heterogeneous streams where self-distillation is not.

Continual and test-time learning. Our setting is continual learning without task boundaries (Parisi et al., 2019) combined with test-time training (Sun et al., 2020); unlike classic CL we assume verifiable feedback, and unlike TTT the adaptation is permanent and cumulative.

6 Conclusion

We presented an online self-distillation method that turns sparse verifiable feedback into permanent model improvement: verified self-generated answers serve as hints for a self-teacher, a reward-filtered memory scaffolds retrieval and training, and forward re-evaluation converts weight gains back into data. The method wins on all seven StreamBench tasks, absorbs a three-domain distribution shift that collapses an RL baseline—with one shared model beating per-domain specialists via weight-borne transfer—and outperforms batch training with the identical loss on identical data. **[TODO: Pending: online-vs-batch transfer to an unseen domain; batch with multi-sample exploration.]** Limitations include the extra oracle calls spent on re-evaluation, single-model experiments at 7B scale, and binary-feedback tasks; richer textual feedback and larger scales are natural next steps.

References

- Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. ICLR, 2024.
- Arash Ahmadian, Chris Cremer, Matthias Gall , Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet  st n, and Sara Hooker. Back to basics: Revisiting REINFORCE style optimization for learning from human feedback in LLMs. ACL, 2024.
- Jian Hu. REINFORCE++: A simple and efficient approach for aligning large language models. arXiv preprint arXiv:2501.03262, 2025.
- Jonas H botter et al. Reinforcement learning via self-distillation (SDPO). arXiv preprint arXiv:2601.20802, 2026.
- Aman Madaan, Niket Tandon, Peter Clark, and Yiming Yang. Memory-assisted prompt editing to improve GPT-3 after deployment. EMNLP, 2022.
- German I Parisi, Ronald Kemker, Jose L Part, Christopher Kanan, and Stefan Wermter. Continual lifelong learning with neural networks: A review. Neural Networks, 2019.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, et al. Deepseek-math: Pushing the limits of mathematical reasoning in open language models. arXiv preprint arXiv:2402.03300, 2024.
- Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. In ICML, 2020.
- Cheng-Kuang Wu, Zhi Rui Tam, Chieh-Yen Lin, Yun-Nung Chen, and Hung-yi Lee. Stream-bench: Towards benchmarking continuous improvement of language agents. In NeurIPS Datasets and Benchmarks, 2024.
- Zhaorui Yang, Tianyu Pang, Haozhe Feng, Han Wang, Wei Chen, Minfeng Zhu, and Qian Liu. Self-distillation bridges distribution gap in language model fine-tuning. ACL, 2024.

A Leakage Audit

We verify four causal-ordering invariants programmatically over the raw run logs of the fused experiment (script released with the code): (A) every problem is scored exactly once; (B) no memory entry predates its problem’s scoring batch; (C) memory is append-only with respect to the stream, so retrieved demonstrations always come from strictly earlier batches; (D) aggregate counters match the per-problem log. All checks pass. Gold labels exist only inside the 0/1 oracle; the stored hint is always the model’s own earlier oracle-passing output. [TODO: Include example student/teacher prompt traces (available in the repo’s leakage-audit folder).]

B Hyperparameters and Reproducibility

[TODO: Full hyperparameter table (LoRA config, lengths, decoding, trainer settings), hardware (single H200 per run), and per-benchmark oracle definitions. All code, per-problem CSVs, and analysis scripts are in the released repository.]

C Additional Ablations

[TODO: Full ablation tables: generate-from-teacher (standalone + fused), epoch/learning-rate grid, multi-sample distillation (filtered and unfiltered $N=10$), REINFORCE++ collapse diagnostics (KL/clip-fraction/advantage-std trajectories).]